D0020E: Project report

Group <3>: <PEPPOL Arrowhead-Based Solution for Automated Order and Invoice Handling>

Isak Hansson	sikhan-3@student.ltu.se
Abdulsalam Alkalash	abdala-3@student.ltu.se
Johan Hynell	johhyn-2@student.ltu.se
Ask Frössling	askfrö-2@student.ltu.se

March 8, 2026

Contents

1 Introduction **3**
1.1 Background . . . 3
1.1.1 Why Peppol . . . 3
1.2 Problem description . . . 3
1.3 Tasks and goals . . . 3

2 System design **4**
2.1 Front end design . . . 4
2.2 GUI implementation . . . 6
2.2.1 Overall architecture . . . 6
2.2.2 Purchase Order Form . . . 8
2.2.3 Future improvements . . . 8
2.3 User stories . . . 8
2.3.1 Human user stories (GUI) . . . 8
2.3.2 Automated service stories (Arrowhead API) . . . 9
2.3.3 Administration and test stories (deployment) . . . 9
2.3.4 Scope note . . . 10
2.4 Peppol implementation . . . 10
2.5 Back end design . . . 10

References **11**

Appendices **12**

A Instructions for installing, testing and source code **12**

1 Introduction

1.1 Background

Many industrial and organizational environments require the regular exchange of purchase orders and invoices. Traditionally, these processes have often been handled manually, for example, through email or paper-based workflows. As systems and organizations increasingly move towards automation, however, manual handling becomes inefficient and difficult to scale. Within the European Union, the standard approach for the electronic exchange of such business documents is PEPPOL.
PEPPOL (Pan-European Public Procurement Online) is a specification and network designed for the standardized exchange of business documents, including orders and invoices. It provides a common format and communication infrastructure that enables secure, interoperable document exchange between organizations across national and organizational boundaries.
Eclipse Arrowhead is a framework for developing systems of systems with a focus on automation and digitalization in IoT environments. The framework aims to achieve interoperability between heterogeneous systems. It does this by providing core services for the discovery, orchestration, and authorization of secure services. This makes Arrowhead suitable for building distributed and autonomous system architectures.
Synecdoque's mbaigo is an implementation of the Eclipse Arrowhead framework that emphasizes security, modularity, and ease of deployment. While the Synecdoque systems collection provides functionality for internal handling of orders within an Arrowhead-based environment, it currently lacks support for sending and receiving orders and invoices using a standardized external format such as PEPPOL. This limitation motivates the development of an integration that enables standardized business document exchange between Arrowhead systems and external service or product providers.

1.1.1 Why Peppol

PEPPOL is selected because it enables the automated exchange of orders and invoices in a standardized and machine-readable format, reducing the need for manual handling. As industries increasingly move towards fully automated and interoperable systems, manual processing of business documents becomes inefficient, error-prone, and difficult to scale. PEPPOL is widely adopted within the European Union and supported by both public and private organizations, making it a practical choice for cross-organizational communication. By using PEPPOL, the proposed Arrowhead-based solution can interact with external partners in a secure, compliant, and future-proof manner.

1.2 Problem description

A standardized way to send and receive orders and invoices is needed in the Synecdoque mbaigo implementation of Eclipse Arrowhead. Without such a mechanism, all outgoing and incoming orders and invoices must be handled manually. Doing so is inefficient and limits the level of automation that can be achieved. This becomes a significant problem in system-of-systems environments, where services are expected to interact autonomously and across organizational boundaries.
By using the European Union standard (PEPPOL), the system can exchange business documents with organizations in up to 98 countries using a common and well-defined format. This enables interoperability between independent systems and reduces the need for custom integrations. The anticipated benefit of the project is to automate parts of the order and invoice handling process, thereby reducing human involvement in repetitive administrative tasks while improving scalability and reliability.

1.3 Tasks and goals

The project task is to implement an Arrowhead service using mbaigo that supports sending and receiving orders and invoices through PEPPOL. The system shall provide an API that can be used by other services

within the Arrowhead network, enabling automated order handling without human intervention. In addition, a web-based interface shall be developed to allow manual management of orders and invoices when required. The overall goal of the project is to demonstrate a working integration of PEPPOL within an Arrowhead-based architecture. Successful completion of the project is defined by the ability to create, send, and receive standardized orders and invoices, both through automated service interactions and through the web interface. The scope of the project is limited to demonstrating core functionality rather than providing a full-scale production-ready solution.

2 System design

2.1 Front end design

For the graphical user interface, we took large inspiration from the mail inbox, something which most people are already familiar with, decreasing the learning curve for the user. Below is on of the earlier GUI prototypes

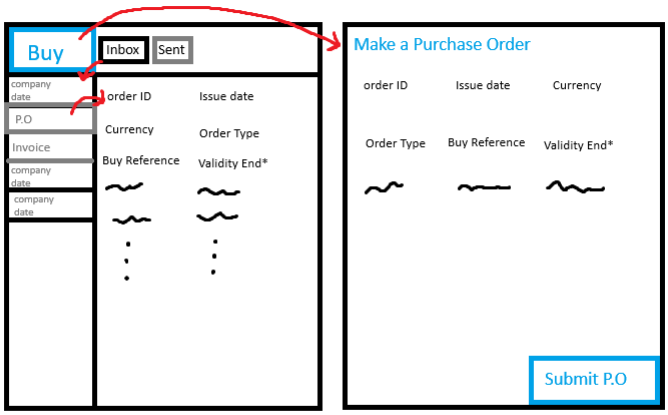

Figure 1: Initial design prototype of GUI

This prototype is similar to a mail inbox in that business exchanges (purchase orders and invoices) are displayed in a list on the left-hand side, similar to emails being received. There is also a large button in the top left, meant to compose a purchase order, similar to writing a new email. Further to make a purchase order we made this sequence diagram to show how the system interacts with the peppol service.

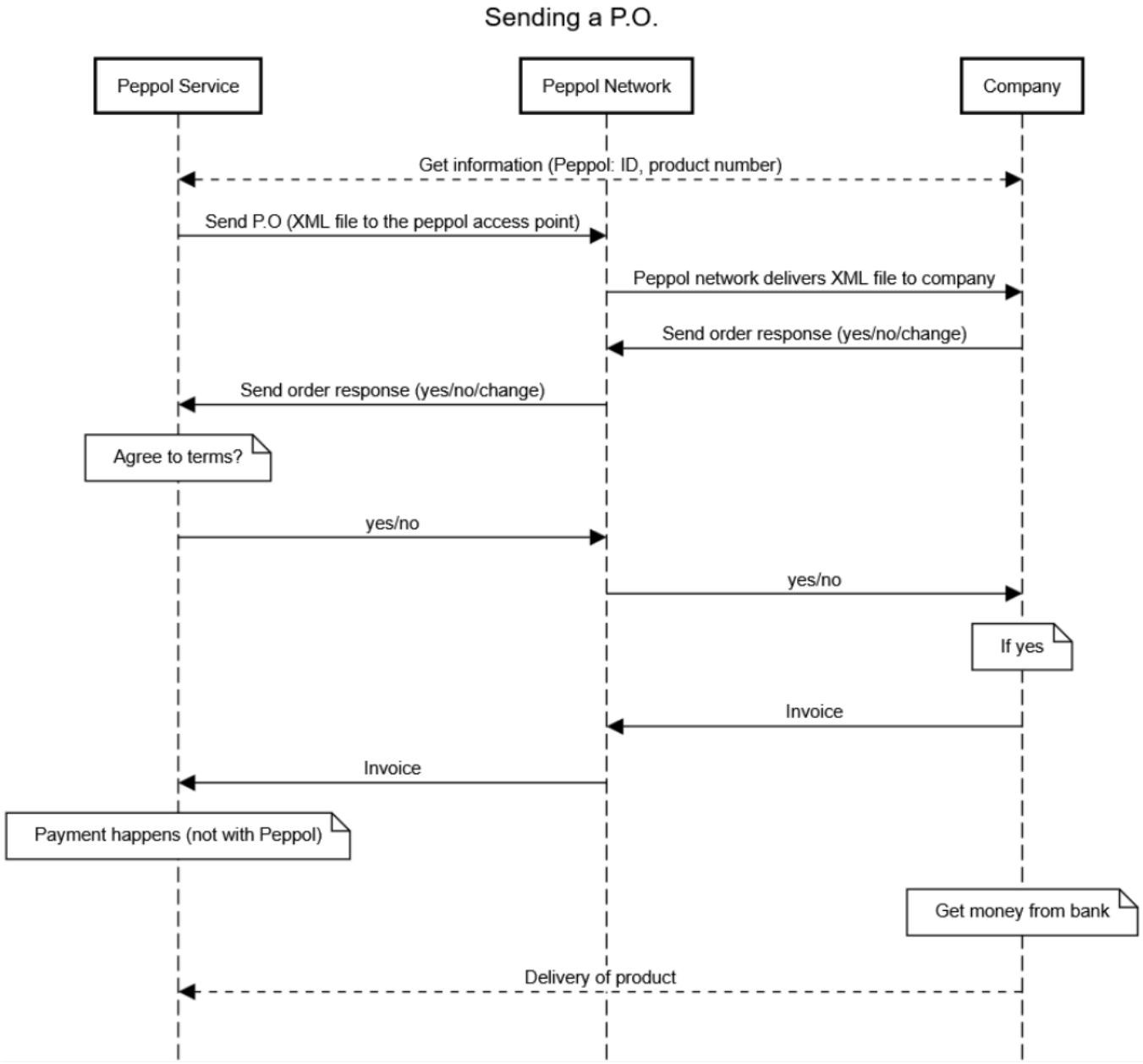

Figure 2: Sequence diagram of making a purchase order

Notably, if we were to implement a way of handling the order response, which is not required but could be seen as part of the MVP; since it is customary these days—there would need to be buttons allowing the user to choose either yes or no to the proposition. This could, for instance, look like this.

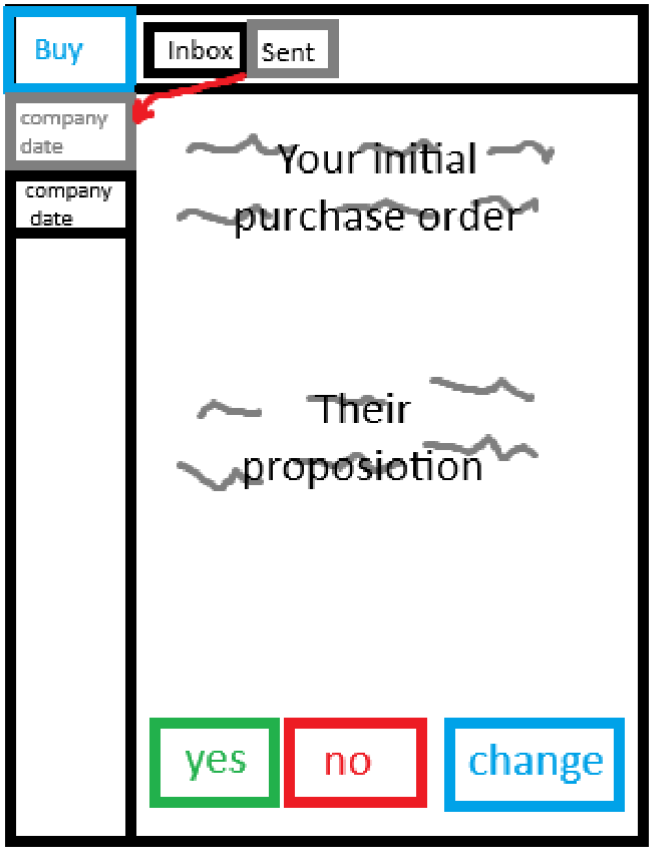

Figure 3: Initial design prototype of GUI

Both parties must agree on the written terms. As the user, you have the option to agree to the terms from the other party, cancel the order completely, or make a counteroffer as many times as needed. Once both parties have agreed and pressed "yes," the exchange will be generated in the Inbox tab, showing the final agreed-upon purchase order and the invoice sent later.

2.2 GUI implementation

The graphical user interface (GUI) of the Peppol Service system was developed as a single-page application (SPA) using **React**[1], chosen for its component-based architecture, strong ecosystem, making it easier to develop dynamically changing interfaces. The frontend is built with modern JavaScript XML (JSX), and CSS for styling. React offers a reactive user experience where the system only updates the elements necessary making it very efficient for heavy workloads, something which could happen if a user has many documents loaded.

2.2.1 Overall architecture

The application follows a tab-based navigation pattern based on the familiar mail inbox with three main views:

- **Inbox**2.2.1 — displays finalized business exchanges
- **Exchanges**2.2.1 — shows ongoing negotiation threads

- **Purchase Order Form**2.2.1 — a forum with multiple fields for creating new purchase orders or counter-proposals

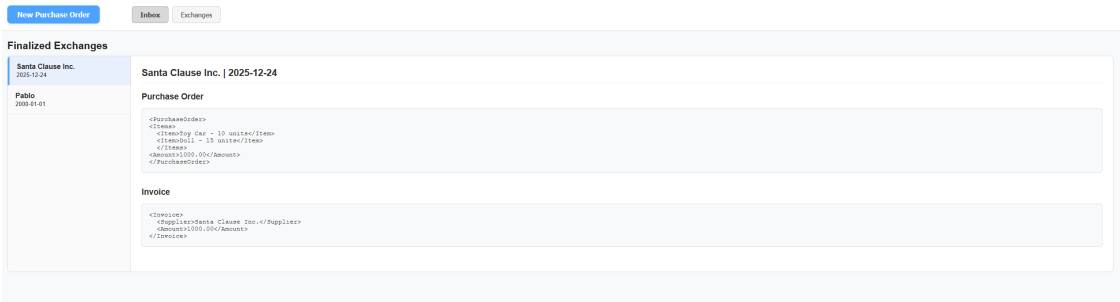

Figure 4: View of the Inbox tab

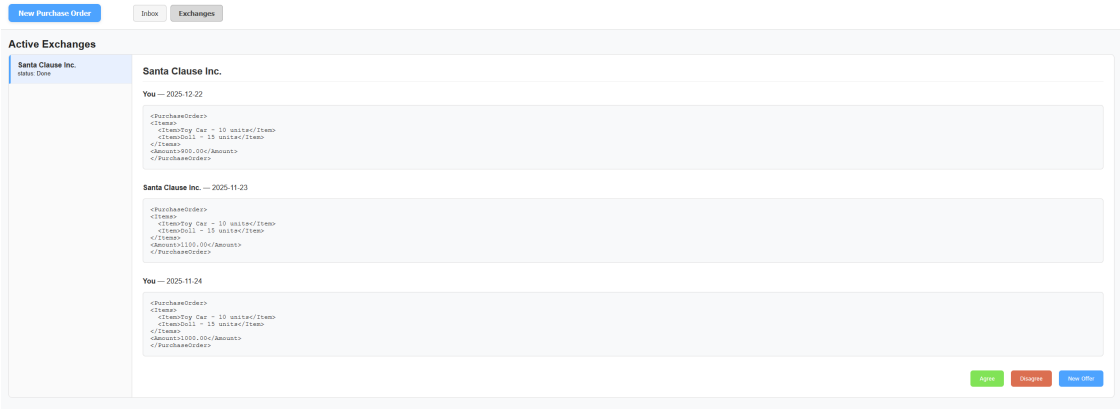

Figure 5: View of the Exchange tab

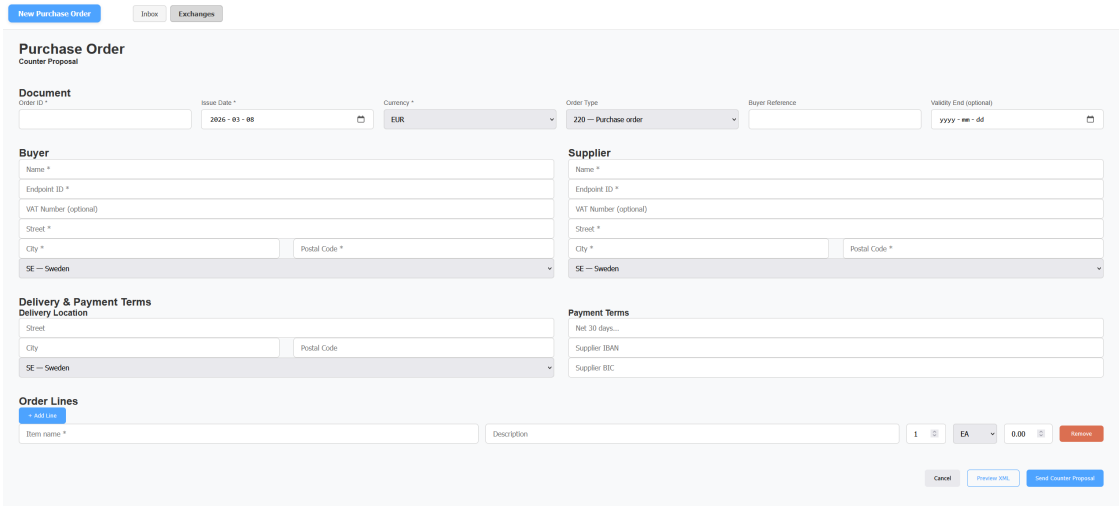

Figure 6: View of the Purchase Order tab

The central component `App.jsx` (or `App.tsx`) manages global state using React hooks:

- `activeTab` — controls which view is visible (`'inbox'` or `'exchange'`)
- `showForm` — toggles visibility of the purchase order form
- `formContext` — carries context information (`mode`: `'new'` or `'counter'`, `threadId`)

2.2.2 Purchase Order Form

The form is a full Peppol xml ready format with placeholder variables for input, key implementation features include:

- **XML generation**: one large form object for document metadata that is later used to search and replace with what the user inputs in the fields in the `Purchase Order` tab, this makes it easy to modify the XML file.
- **Dynamic lines**: users can add/remove order lines (item name, description, quantity, unit, price), this is later iterated over to include every item in the purchase order.
- **Preview functionality**: a modal window shows the generated XML before submission
- **Context awareness**: the context of where the user comes from is stored in the meta data to ensures the xml file gets placed properly in the database.

2.2.3 Future improvements

- **API fetching and posting**: ones the database is up and running the and API endpoints are declared the systems can start using persistent storage that the user can manipulate through the frontend GUI
- **Sending and receiving ACK**: implement another form for when the user clicks the `Agree` button in the `Exchange` tab, shows that the user agrees to the terms of the purchase order
- **Sending NACK**: same reasoning as above just that the user digress with the terms of the purchase order and wants to cancel the deal entirely

2.3 User stories

To ensure that the system requirements are captured from a practical user perspective, we describe the intended functionality using user stories. These stories complement the use case and sequence diagrams by specifying *who* interacts with the system, *what* they need to achieve, and *why* the functionality is valuable. In our project, the user stories cover both manual operation through a web-based GUI and automated interactions through Arrowhead services, reflecting the goal of reducing human involvement in routine order and invoice handling.
The project is intended to support both manual and automated handling of purchase orders and invoices in an Arrowhead-based environment. The following user stories capture the expected behaviour from the perspectives of human users, automated services, and system administrators.

2.3.1 Human user stories (GUI)

1. **Create purchase order (buyer)**: As a buyer, I want to compose a purchase order in the GUI, so that I can send standardized procurement requests without manual document formatting.
    - Acceptance: The GUI provides a “Compose” action and a structured form for buyer/supplier identifiers, order lines, and basic terms.

- Acceptance: The user can preview the outgoing document before sending.

2. **Inbox overview (buyer and supplier)**: As a user, I want an inbox-style view of business exchanges, so that I can quickly see incoming and outgoing orders and invoices in one place.
   - Acceptance: Exchanges are listed with key metadata (counterparty, date, status).
   - Acceptance: Selecting an entry shows the full details of the order or invoice.
3. **Respond to an order (supplier)**: As a supplier, I want to respond to a received purchase order (accept, reject, or propose a change), so that negotiation can be handled in a structured way.
   - Acceptance: The GUI provides clear actions such as *yes/no/change*.
   - Acceptance: The response is stored and reflected as an updated status in the exchange thread.
4. **View final agreement (both parties)**: As a user, I want to see the final agreed purchase order after negotiation, so that both parties share the same authoritative version of the agreement.
   - Acceptance: The final document is shown as the latest accepted version in the exchange.
   - Acceptance: Earlier proposals remain viewable for traceability.
5. **Receive and read invoice (buyer)**: As a buyer, I want to receive invoices related to purchase orders and view them in the GUI, so that invoicing becomes part of the same workflow as ordering.
   - Acceptance: The invoice appears in the exchange thread and can be opened and read.
   - Acceptance: The invoice is linked to its originating purchase order.

2.3.2 Automated service stories (Arrowhead API)

1. **Submit order via API**: As an internal Arrowhead service, I want to submit a purchase order through an API, so that order creation can be automated without a human user.
   - Acceptance: The API accepts structured order data and returns a unique identifier and initial status.
2. **Query status via API**: As an internal Arrowhead service, I want to query order and invoice status, so that downstream automation can react to acceptance, rejection, or invoicing events.
   - Acceptance: The API supports retrieval of current status and basic metadata by identifier.
3. **Receive invoice via API**: As an internal Arrowhead service, I want to receive invoice notifications programmatically, so that invoicing can trigger further automated processing (e.g., accounting or approval workflows).
   - Acceptance: The API exposes invoice retrieval (pull) or notification (push) functionality within the local system-of-systems.

2.3.3 Administration and test stories (deployment)

1. **Configure endpoints and identities**: As a system administrator, I want to configure identities and endpoints (e.g., organization identifiers and access point settings), so that the system can communicate correctly in the intended network environment.
   - Acceptance: Configuration is externalized and can be changed without code changes.
2. **Local PEPPOL simulation for testing**: As a developer/tester, I want to run a local test setup that simulates PEPPOL communication, so that development and verification can be performed without real organizational registration.

- Acceptance: The system can exchange documents between two locally configured access points in a controlled environment.

2.3.4 Scope note

The user stories above define the intended MVP behaviour: creating and viewing exchanges, sending purchase orders, handling responses, and receiving invoices through a combination of GUI interaction and internal service APIs. Advanced features such as full financial payment processing are considered out of scope for this project and are treated as future work.

2.4 Peppol implementation

The PEPPOL network is accessible only to organizations and not private individuals. To test our implementation, we will instead simulate a small PEPPOL network locally using Oxalis (an open-source implementation of PEPPOL access points). Our local testing network consists of two access points that can communicate with each other, each can have applications attached that want to communicate via PEPPOL.
(image of diagram: application<->AP1<->AP2<->other application)

2.5 Back end design

(Ask Skriver här)

References

[1] *The library for web and native user interfaces.* URL: `https://react.dev/`.

Appendices

A Instructions for installing, testing and source code

More information about the project and the instructions for installing, testing and recommended further development can be found on the **Github repository**. All source code for the project is also found there.